WASP SE Assignment – Antonio Napoli (KTH) October 2026

Introduction

My research project presents a novel approach to determine when two machine states are observationally equivalent by synthesizing a projection function (Observation) using the Syntax-Guided Synthesis (SyGuS) methodology and cvc5 in order to create a leakage model aggregating multiple observations in many iterations. Two machine states are observationally equivalent if an attacker observing the execution of both states cannot distinguish them. We explore the problem of observational equivalence in machine states consisting of 64-bit vector registers, with examples initially derived from a ground truth model representing the ideal leakage. The goal is to improve the efficiency of the synthesized projection function, which distinguishes equivalent and non-equivalent states after executing specific instructions. Our main contributions are : optimization of the synthesis with the help of large language models (LLMs), by extracting constants and subexpressions from CPU technical documentation to augment SyGuS grammars; refinement of the observation synthetized, generating example that invalidate the observation with respect to the ground truth; reuse part of the observation found as subexpression to enrich the grammar. This enables enumeration-based synthesis that avoids SMT-based constant generation and optimizes the process. Queries are instruction-dependant and CPU-dependent, with the LLM providing constant values and sub-expression that reveal operand-dependent leakage (e.g., ARM7TDMI multiply instruction) or memory-dependant leakage (Tag-idx). The area touched by this project are multiple : Usage of LLM, software side-channel detection, verification, SMT solving and optimization. The main focus is put in the optimization area, since the goal aim to detect leakage previously undetected due to high computational cost. The usage of LLM is mainly from a user application side, where we use tecnique as RAG, reasoning and task splitting in order to reduce the search-space of the SAT solver, crafting a CPU tailored grammar. The security of side-channel while it's the main reason that led to this work, it's somehow marginal and we reuse methods and definition previously presented in [4].

1 Lecture principles

1.1 Verification versus Validation (Lecture 2)

The lecture covered the difference between verification which checks that we build the product right, and validation which checks that we build the right product. In my project, I can see these two aspects separated by the ground truth model. The end goal of the project is to reduce complexity with an ML approacj and increase efficiency, as mentioned: build the product right; but since it's a verification process, and it involves the detection of an information leakage, using a sat solving tool, we ensure that our output is reliable to respect of the data taken as input, verifiable and reporducible. We add an ulterior task at

the end of the process with multiple test, where the data are SAT generated, to measure the soundness and the precision of our formula, in order to check if we overapproximated or underapproximated in our output. This is precisely the verification part: the solver ensures that the synthesized observation agrees with the ground truth at the examples, and that the final result over- or under-approximates the ground truth. What the solver cannot do is confirm that the ground truth observation correctly captures the leakage of the real CPU. As explained in the lecture, ML is similar to requirements engineering in that both attempt to induce rules from examples, and synthesis from examples is akin to inductive reasoning. This implies that our specification is partially learned, and thus the validation part is even more important than usual in verification. One possible next step is to evaluate the synthesized observations against timing measurements on a real CPU or another model.

1.2 Exploring Disconfirmatory Evidence (Lecture 3)

The lecture on behavioral SE highlighted confirmation bias, the tendency to seek evidence in favor of one's hypothesis, and one approach to combating it was precisely to seek disconfirmatory evidence. The refinement step of our pipeline accomplishes something similar to that, but in an automated way: instead of looking for examples that confirm the observation, we search for states that contradict it with respect to the ground truth, similar to how property-based testing looks for counterexamples to a property. However, there are other ways in which confirmation bias might have affected our work. The constants and subexpressions suggested by the LLM guide the grammar to favor expressions present in the documentation, and I myself am biased toward examples where I suspect leakage to occur. One way to reduce confirmation bias in this pipeline would be to run the synthesis without the LLM suggestions as a baseline and track what rejections occur during refinement.

2 Guest-Lecture Principles

2.1 Requirements Engineering (Julian Frattini)

Julian Frattini, Ph.D, emphasized in his lecture the importance of requirement engineering. This usage of the LLM encoded in out project also display the need of ML engineering in the building process. Even if the system used is OpenAI API, and the implementation is rather easy compared to build an entire Neural Netowrk or other complext ML tecniques, we noticed that in order to not slow down the evolution of the project on the science side it was needed to build a reliable integration. With the change of the idea -scientifically talking- it was needed to re-structure the whole implementation since it was very hard scoped, furthermore we decided to create a standalone tool for this in order to use it and integrate in other process used by other researcher. Concretely, we never fixed the format of what the LLM returns, nor the interface between the LLM module

and cvc5, nor when an observation is considered good enough. Whenever the scientific idea changed, these had to be redefined all together. This phase even if critical it was not considered at the start of the project, and since the multiple parts involved in the tasks, this lack of requirement handling became a problem later on. I would argue that usually it might be difficult to write down specific technical requirement in the solution space, since PhD projects have to produce innovation and usually are goal oriented and define in the problem space. It is difficult to know the technical requirement in order to reach the goal, nevertheless trying to sketch a possible idea would surely help and avoid further deviation that might mislead later on.

2.2 Human Aspects in Software and AI Engineering (Per Lenberg)

Per Lenberg stressed the importance of psychological safety and team norm clarity in his research. Our project involves researchers from KTH and CISPA, and having clarity on who is responsible for what and how changes to the plan will be communicated would have improved our collaboration during implementation phase significantly. Psychological safety is also important in research, as it allows to save time spent on wrong ideas earlier in the process. In our case, the shift to the new direction happened too late.

3 Data Scientists versus Software Engineers

In the book "Machine Learning in Production" (Chapters 1 and 2) [3], the author discusses the distinction between Data Scientists and Software Engineers. Specifically, chapter 2 makes the case that the model is only one part in a bigger system, and most of the engineering effort has to be put into everything else besides it. I broadly agree with this difference, because Data Science usually they are mostly focused on the Model-Algorithm scope, and they overlook on other corner technicalities such a depolyment, integration, robustness and so on. Usually these latest are handled by Software Engineer, which, on the other hand, might lack experience in optimizing the performance of a model, while still being able to fully deploy and utilize one. These two roles might converge in the future, and I see more potential in the Data Scientist that might make up their lack of technical programming skill also thanks to the advent of the LLM, which might be able to fully deploy a solution with the right indications given by a Data Scientist. On the other hand I can't see the same takeover from the Engineers, because the theoretical knowledge behind the ML is crucial and I doubt LLM might ever help in optimizing a "new problem". My own project is, in a way, an example of the convergence: I design the synthesis approach and the prompting strategy, but I also had to roll up my sleeves and write some code to implement the integration and a standalone tool, which is the job of a software engineer.

4 Paper Analysis: LLMs for Test Input Generation for Semantic Applications [1]

4.1 Core Ideas and importance to the SE

The core idea of this paper is to introduce a technique to mitigate some errors in LLM system such as false positive caused by the cache reuse. LLM calls are expensive at scale; semantic caches reuse past answers for queries with different phrasing but the same meaning. As it uses query embeddings, it can give a wrong answer to a query that only looks similar, or miss a query that should be a hit. To test it requires labelled pairs of similar queries, which are not generally available. VaryGen uses an LLM to adapt queries to the domain, to generate subtle variations of them, and to evaluate the generated dataset. The authors show that the generated pairs agree with human judgement of similarity and that they expose failures of GPTCache in its default configuration. This is important for AI engineering as semantic components have no exact-match oracle, and test data and the oracle have to be built differently.

4.2 Relation to your research

The relation to my research is in the test input generation aspect. The quality of the inputs directly affects the validity of the synthesized function or observation. Their idea of using LLMs to generate subtle variations of queries it might help with my own use of LLMs to generate constants, subexpressions, and instruction-specific variations for SyGuS grammars. The strongest connection is, however, conceptual: the semantic cache decides whether two queries are equivalent while our observation decides whether two machine states are equivalent. A wrong cache hit corresponds to an observation that over-approximates; it considers two states equivalent, that an attacker can distinguish. In both cases the interesting test inputs are the ones close to the boundary, which is the idea of boundary value testing seen in lecture 2.

4.3 Integration into a larger AI-intensive project

In a larger AI-intensive project such as a Secure CPU Verification Platform, semantic caches might be used for efficiently serving verification queries across distributed nodes. The VaryGen approach could be applied to automatically generate workloads of queries that stress-test the cache before deployment. Such a platform would take an instruction and a CPU model as inputs and produce a leakage model for it as an output. The AI parts would consist of the LLM reading the technical documentation and possibly an assistant explaining the details to the user. My research would serve as the synthesis engine which would produce the leakage model.

4.4 Adaptation of your research

The knowledge gained fromm this paper might help in the future in my own research. Two concrete adaptations come out of this paper. The first is creating near equivalent-state pairs, say, differing by a single operand bit, as a stress test for a synthesized observation in the same way that VaryGen produces near equivalent queries. The second is caching the LLM hints per instruction and CPU and testing that this cache with generated variations of the queries. Since a main part of my project is LLM dependant, and the area is security-formal verification, any improvement would make a difference. Indeed the structure of my work and this are different, such as the main goal and the dataset used, but they have many points in common that could be abstracted and reused into my own research.

5 Paper Analysis: Bringing Machine Learning Models Beyond the Experimental Stage with Explainable AI [2]

5.1 Core Ideas and importance to the SE

This paper presents an analysis done in a Danish Bank, where a ML model was successfully integrated into the day-to-day work and it presents the main reason which made it possible. The authors observe that many ML models never graduate out of a testing environment. In their case study one previously unutilized model was put into a mission critical system, and the critical factor was explainable AI: the explanations increased the confidence of the domain experts and allowed them to validate and retrain the model in production, without being software developers. The findings are based on a structured evaluation and interviews with the expert users. From an SE point of view the paper shows that trust and human oversight are deployment requirements, not only accuracy.

5.2 Relation to your research

The relation is not immediate to my research, but in the broader area of software security. I'm mainly interested in the Explainable AI because in software security it's not enough to know if a piece of code, or an architecture, is vulnerable or not, it's needed to understand what causes the vulnerability.

In this sense our approach is explainable by construction: the synthesized observation is a readable formula over the state, showing for example which bits of the operand drive the timing of the multiply instruction.

5.3 Integration into a Larger AI-Intensive Project

Consider a CI pipeline at a hardware vendor that analyses every new core design, flagging instructions that leak. Following the paper, the flags would only be trusted and acted upon

by security engineers if they come with an explanation of why the instruction leaks. Our synthesized observations would provide exactly these explanations.

5.4 Adaptation of your research

Ad example into my pipeline I do start with an input of data labeled as : information leakage, sound. And the goal of my project is to understand what the cause behind this separation. In a possible future research, I plan on working on using an ML model with explainability. In this way I would cancel the need of a SAT-solver, or at least make it secondary, and focus on performance given by these models. At the state of the art, the main limitation is based on the performance since the generation of more datapoint is negative for a solver, but on the other hand ML is not usable because it would miss the reparation idea. The explainability would be a good idea that would fit both problems.

6 Research Ethics and Synthesis Reflection

n order to ensure originality I did use only the material mentioned by the assignment file, and the papers were found directly on the CAIN website. I wrote without the use of LLM or external tools. I surfed the CAIN main website, and looked into the latest 3 years of paper published. From here I mainly opened papers in the area of LLM and Explainable AI, since these were the topic I was interested in.

6.1 Pitfalls and Mitigations

Some titles were misleading: several papers mentioning LLMs were about general LLM usage, but not the engineering around LLM-based systems. VaryGen paper is also listed under two titles: ”Semantic Caches” on arXiv, and ”Semantic Applications” in the proceedings. After the first attempts I stopped selecting by title only, and instead read the abstract and the session the paper was presented in. I

References

[1] Z. Rasool, S. Barnett, D. Willie, S. Kurniawan, S. Balugo, S. Thudumu, and M. Abdelrazek. LLMs for Test Input Generation for Semantic Applications. In *Proc. 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024)*, pp. 160–165, 2024. arXiv:2401.08138.

[2] N. W. Mikkelsen, L. Pedersen, M. Hussain, V. Foged, and E. Kindler. Bringing Machine Learning Models Beyond the Experimental Stage with Explainable AI. In *Proc. 4th International Conference on AI Engineering – Software Engineering for AI (CAIN 2025)*, 2025.

[3] C. Kästner. *Machine Learning in Production: From Models to Products*. Cambridge, MA: MIT Press, 2025. https://mlip-cmu.github.io/book/

[4] H. Nemati, P. Buiras, A. Lindner, R. Guanciale, and S. Jacobs. Validation of Abstract Side-Channel Models for Computer Architectures. In *Proc. 32nd International Conference on Computer Aided Verification (CAV 2020)*, LNCS 12224, Springer, 2020.